Adaptive Search-and-Training for Robust and Efficient Network Pruning

Xiaotong Lu, Weisheng Dong, *Member, IEEE,* Xin Li, *Fellow, IEEE,* Jinjian Wu, *Member, IEEE,* Leida Li, *Member, IEEE,* and Guangming Shi, *Fellow, IEEE*

Abstract—Both network pruning and neural architecture search (NAS) can be interpreted as techniques to automate the design and optimization of artificial neural networks. In this paper, we challenge the conventional wisdom of training before pruning by proposing a joint search-and-training approach to learn a compact network directly from scratch. Using pruning as a search strategy, we advocate three new insights for network engineering: 1) to formulate adaptive search as a cold start strategy to find a compact subnetwork on the coarse scale; and 2) to automatically learn the threshold for network pruning; 3) to offer flexibility to choose between *efficiency* and *robustness*. More specifically, we propose an adaptive search algorithm in the cold start by exploiting the randomness and flexibility of filter pruning. The weights associated with the network filters will be updated by ThreshNet, a flexible coarse-to-fine pruning method inspired by reinforcement learning. In addition, we introduce a robust pruning strategy leveraging the technique of knowledge distillation through a teacher-student network. Extensive experiments on ResNet and VGGNet have shown that our proposed method can achieve a better balance in terms of efficiency and accuracy and notable advantages over current state-of-the-art pruning methods in several popular datasets, including CIFAR10, CIFAR100, and ImageNet.

Index Terms—Model Compression, Network Pruning, Knowledge Distillation.

1 Introduction

Deep Neural Networks (DNN) have achieved great success in many computer vision tasks, such as image classification, object detection, segmentation, image generation, and object tracking. However, the huge size of the model and the expensive computational cost are key obstacles to the deployment of DNN, especially on some devices with limited computing resources. Consequently, network pruning has been widely used to reduce the high inference cost of deep models in low-resource settings [1]. It is enlightening to think of network pruning as a special case of neural architecture search (NAS), a popular framework for architecture engineering [2]. However, there are two fundamental weaknesses in the existing network pruning approaches. First, the redundancy of computations, i.e., if some filter will be eventually pruned, any related training and updating will be a waste of precious computational resources. Second, the decoupling of pre-training and pruning is likely to produce a suboptimal solution. A more desirable solution is to design DNN through joint optimization of training and pruning.

Based on the motivations described above, we have presented a new *joint search and training framework* to learn efficient compact networks in our preliminary work [3]. Under the proposed joint search and training framework, we have developed a new algorithm to efficiently learn compact networks (see **Figure 1**). In our approach [3], we alternate between two phases: sampler and updater. In the sampling stage, we search for a compact network by weight calculation and thresholding; in the updating stage, the parameters and weights of the trained compact network are mapped back to the target network. Meanwhile, flexible thresholding and random perturbation strategies are introduced to further improve the efficiency of the sampler [4]. After iterative search and training, we select the best compact networks and fine-tune them until convergence. We have validated the proposed method on both ResNet and VGGNet and several well-known datasets (CIFAR-10, CIFAR-100, and ImageNet).

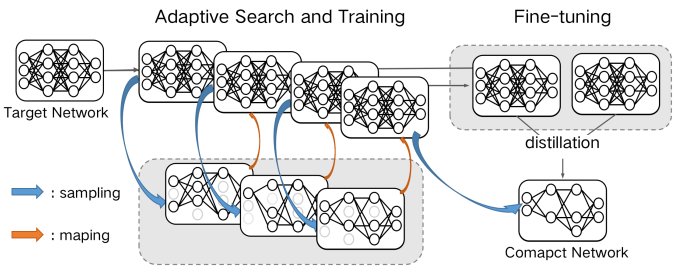


Fig. 1: Summary of the proposed iterative adaptive search and training method. After a coarse training, we first sample compact subnets from the target network using an adaptive search based on the weights; then we train the subnets and map the trained subnets to the target network. This search-train-map process will be repeated iteratively until the sampled subnets converge. The best performing subnetwork will be further fine-tuned by multi-teacher knowledge distillation into the final output pruned network.

There are two new insights brought about by this extended study. On the one hand, to further reduce computational effort and reliance on pre-trained models, we propose an adaptive search-and-training pruning framework in combination with network architecture search methods like DARTS [5]. This is conceptually

This work was supported in part by the Natural Science Foundation of China under Grant 61991451 and Grant 61836008. Xin Li's work is partially supported by the NSF under grant IIS-2114664, CMMI-2146076, and the WV Higher Education Policy Commission Grant (HEPC.dsr.23.7).

X. Lu, W. Dong, J. Wu, L. Li, and G. Shi are with the School of Artificial Intelligence, Xidian University, Xi'an, China (email: xiaotonglu47@gmail.com; wsdong@mail.xidian.edu.cn; Jinjian.wu@mail.xidian.edu.cn; ldli@xidian.edu.cn; gmshi@xidian.edu.cn).

X. Li is with the Lane Dept. of CSEE, West Virginia University, Morgantown, WV26506-6109, USA(email:xin.li@1eee.org).

similar to the exploitation-exploration trade-off in reinforcement learning that has recently been applied to BERT compression [6]. On the other hand, we hypothesize that the joint optimization problem of training and pruning is nonconvex, e.g., the linear combination of two optimally pruned networks is unlikely to generate another valid solution. This observation inspired us to use the effective *deterministic annealing* strategy to tackle the problem of efficient network pruning in a progressive way.

In this extended study, we introduce *three new insights* on the emerging framework of joint research and pruning. First, we propose to expand the search space for network pruning by associating each filter with a learnable weight. Similarly to Transformable Architecture Search (TAS) [7] and DARTS [5], we find a compact network by calculating the distribution of the corresponding weights. Second, we used several large networks together as a teacher network for knowledge distillation [8] to address the challenges arising from the absence of a pre-trained network. Finally, we propose iteratively updating the network architecture from coarse to fine to maximize learning efficiency (conceptually similar to progressive NAS [9]). Combining the NAS strategy with iterative training, it is possible to obtain highly competitive compact networks from scratch in only a few hours instead of days. The technical contributions of this paper can be summarized in four ways:

- We propose a new paradigm of pruning to obtain compact networks from scratch without the need for well-pretrained large networks.
- By jointly implementing adaptive search and training, we can simultaneously reduce memory usage, accelerate learning, and achieve improved pruning efficiency.
- Consistent with our initial pruning strategy, we propose to use multiple teacher networks for the distillation of knowledge, which can effectively improve the robustness of network pruning.
- Extensive experiments have been conducted to show that our method can achieve better accuracy results and shorter wall clock running times compared to other competing methods.

Note that the preliminary conference version of this work appeared in [3]. The conference version did not analyze the learning efficiency-accuracy trade-off and only briefly investigated the correlation between pruning operations and weights. Therefore, in this extended version, we analyze more efficient pruning strategies and present more robust pruning results. Additionally, the analysis of the channel selection criterion is a separate contribution that has not appeared in our conference version.

The remainder of this paper is organized as follows. Sec. 2 of this manuscript includes existing work related to this article. Sec. 3 presents the proposed network pruning method. Sec. 4 shows the simulation results of the proposed method with comparisons with other competing methods. Finally, Sec. 5 presents the conclusions and future directions of the pruning technique.

2 Related Works

In this section, we review the papers relevant to model compression, covering various methods for network pruning, as well as network architecture search, lightweight network design, and knowledge distillation.

Unstructured Pruning methods aim to enforce the sparseness of the filters or channels in large networks. According to the theory of sparse representation, the pruned network after trimming the sparse terms will have a distribution similar to that of large networks without pruning. To optimize the number of neurons and the number of parameters, an early method [10] incorporates the sparse constraint into the objective function and decimates the number of neurons during training. Another approach [11] proposes a low-rank approximation by replacing a base of linear combinatorial convolutional kernels of convolutional kernels $N \times N$ with a decomposed number of convolutional kernels $1 \times N$. The regularization of the group sparsity in the network parameters is proposed in [12], where each group is defined to act on a single neuron. More recently, an efficient network pruning algorithm is proposed in [13], which can prune the network in a few minutes. Group Lasso is used in [14] to sequentially achieve the speedup of network prediction. This is a ℓ_1-like regularization approach, which can be seen as treating weights as groups and achieving structured sparsity for the learned parameters. However, this method has little advantage in performance due to its manual design and dependence on the well-trained target network. Although unstructured pruning methods can achieve a high compression ratio due to the smaller granularity of pruning, they are not suitable for hardware devices [15]. To avoid this drawback, recent work [16] and [17] opt to combine unstructured pruning with structured pruning at the convolutional kernel level. Among the latest advances, discrimination-aware channel pruning (DCP) [18] was formulated as a sparsity-inducing optimization problem with a convex objective; The loss-metric mismatch problem was tackled using a performance maximization approach in [19].

Structured Pruning methods iteratively prune and tune filters or layers in the network to reduce the performance loss associated with handcrafted design. A typical structured pruning paradigm is to first train a large target network, then prune, and finally fine-tune the compact network. The objective of pruning is to remove relatively unimportant filters and combine the remaining filters into a compact network. Mainstream pruning methods focus on how to prune properly under the framework of training-before-pruning. For example, in [20], the authors switch from pruning contributions that are relatively small to pruning contributions that are most substitutable. When computing the geometric median for different filters in the same layer, the closer the geometric center, the easier it is to be represented by other filters. In [21], structural regularization is applied to mutually corresponding out channels and channels in the continuous network layer, which can remove more redundant channels with a smaller accuracy loss. The work of [22] provides more room to optimize the representation of the network and the ability to perform tasks while retaining the pruned filter. Along this line of research, a continuous pruning strategy is proposed in [7], where the pruned structure is treated as the target of the search and NAS is applied to simultaneously search for the optimal depth and width of the network. Another approach [23] automatically prunes each layer of the network based on meta-learning and fine-tunes the automatically learned pruned network. EagleEye [24] exploits the strong correlation between the different pruned DNN structures and the final precision determined to perform efficient pruning without fine-tuning. More recent work such as [25] and [26] considered sample and network complexity constraints in dynamic or two-stage pruning to achieve better performance. The latest advance ResRep [27] reparameterizes a CNN in memory parts and forgotten parts, and the reduction of structural redundancy (SRR) [28] prunes the filters in the selected layers with maximum redundancy.

Lightweight Network Designing methods avoid pruning by building a simpler network structure than standard networks while achieving the goal of reducing the parameters and calculations of the network. The most popular lightweight network design methods are MobileNet and ShuffleNet. MobileNetV2 [29] achieves a lightweight network design by replacing a standard convolution with a separable convolution by depth, which contains a convolution by depth and a convolution by 1×1. ShuffleNet [30] uses two operations: point-wise group convolution to reduce computational complexity and channel shuffle to facilitate information flow. For a given computation budget, ShuffleNet allows more feature maps to help encode more information on small networks. In addition to network design, the Squeeze-and-Excitation (SE) kernel is proposed in [31] to model the correlation between feature channels. SEnet can learn the weights of the features based on the loss, which in turn makes the weight of the effective feature map heavier and the weight of the ineffective feature map smaller. A more recent work [32] proposes a new ghost module designed to generate more feature maps using inexpensive operations. Based on a set of original feature maps, a series of linear transformations are applied to generate "phantom" feature maps to uncover the desired information from the original features at a small cost. Thanks to the plug-and-play property of ghost modules, one can build a lightweight neural network GhostNet simply by stacking several ghost modules.

Network Architecture Search methods aim to find a potentially optimal network structure from hyperparameterized networks. Although these algorithms have made special efforts to improve efficiency, they generally require prohibitive computing resources, especially for methods based on evolutionary algorithms (EA) [33] and reinforcement learning (RL) [34], [35]. The gradient-based approach [5] relaxes the search space to a continuous spatial structure as a way to allow performance optimization using gradient descent, which can search for a high-performance framework with complex graph topology in a rich search space. However, the number of all candidate filters is fixed to make the loss backpropagated, which also leads to bloating of the original search space. In [36], the authors propose the concept of one-shot search, which greatly speeds up the architecture search by evaluating candidate architectures in the validation set with a pre-trained model and then retraining it. Furthermore, in [37], memory is reduced by binarizing the paths to enable direct training on large data, which eventually allows the network structure to be searched directly on the target task. Although these algorithms based on adaptive learning greatly speed up the parameter-sharing training process, training an overparameterized network is still a heavy burden [38]. To overcome this weakness, a supernetwork that includes different structures is trained in [39] so that a subnetwork can be sampled for a specific deployment scenario.

Knowledge Distillation methods are the transfer of knowledge from a large pre-trained teacher model to a lightweight student model by mimicking the soft labels generated by the teacher network. It has been widely used in model compression because of its good generalizability to lightweight models. In pioneering work on knowledge distillation [8], the authors first trained a teacher network and then used the output of the teacher network as the target of the student network. In [40] a multi-teacher extension is proposed in which multiple teachers work together to guide the learning of the student network. The fixed one-way conversion idea (teacher-student) is further improved by deep mutual learning (DML) [41] - that is, allowing a group of student networks to learn from each other during the training process. Note that it is not the case that the better performing teacher necessarily distills better students due to a potential capacity mismatch (i.e., the student model cannot well mimic the teacher). Based on this observation, an early stop teacher regularization for distillation is proposed in [42] to stop distillation early before reaching the convergence. To efficiently learn the similarity between classes and prevent the student network from overfitting, additional semantic constraints are imposed on the distillation process [43] with the assumption that controlling strictness in training the teacher network can educate better students. With the help of knowledge distillation, many model compression algorithms can often perform knowledge distillation by treating the original network as a teacher network and the compact network after pruning as a student network, which further brings significant improvement. The latest advances include a new framework called network adjustment [44], which considers network accuracy as a function of FLOPs, which is orthogonal to our proposed approach.

Although knowledge distillation can be applied effectively to fine-tune the model after pruning [7], [45], our method [46] cannot directly select teacher networks for distillation due to the absence of pre-training. Although in the improved version, inspired by [47] and [48], these methods use multiple teachers to integrate knowledge from different sources to improve student performance, combined with the proposed joint search training framework, we propose to select multiple large networks as teachers during training. However, unlike [47], [48], we choose a network of teachers that share parameters and, therefore, have some correlation.

Joint Prune-and-Search methods. Recently, inspired by the success of NAS, algorithms that implement pruning using search operations have started to emerge. These algorithms [7], [49] replace the manual design component in traditional pruning with adaptive search. For example, Automl for model compression (AMC) [49] uses reinforcement learning techniques to search for optimal compression strategies when automating compression models; and transformable architecture search (TAS) [7] searches for pruned subnetworks of different depth/width in the search space to explore the optimal pruning scale. A joint search for the network architecture, pruning and quantization (APQ) policy was recently proposed in [50]. In our own work [3], we propose a joint search and training approach by alternating between two phases: sampler and updater.

Similarly, [44] is also a network tuning framework that can maximize FLOP utilization, which is characterized by dynamically adjusting (rather than reducing) the number of channels in the network through an iterative process. When the accuracy is directly treated as a function of FLOPs, the optimized accuracy and the corresponding structure can be obtained under limited FLOPs. However, unlike the search-based pruning method proposed in this manuscript and similar TAS, AMC, etc., [44] is more like a NAS method that searches for the number of channels on a fixed topology. The reason is that [44] only focuses on FLOPs and deliberately ignores the number of parameters, so it tends to reduce the number of FLOPs and increase the number of parameters to improve accuracy, while pruning reduces both the number of FLOPs and the number of parameters.

In this work, we significantly extend this line of research by analyzing the efficiency-accuracy tradeoff, as well as the correlation between pruning operations and weights.

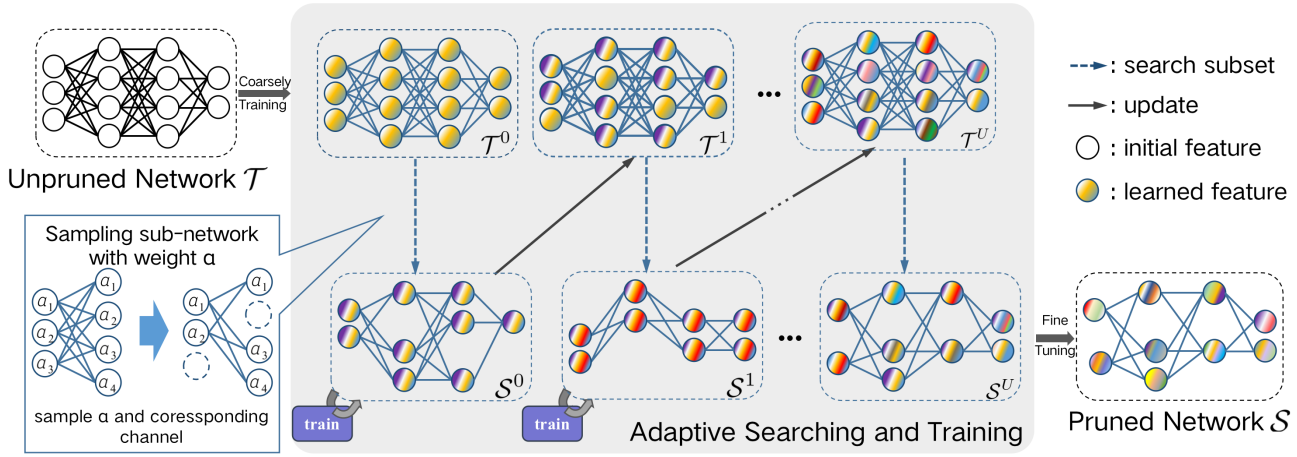


Fig. 2: Detailed illustration of the adaptive search-and-training method. First, we sample compact subnets of the target network based on weight α. Second, we search, train, and map the subnets to the target network. Note that such a search-train-map process will be iterated until the sampled subnets converge. The best performing subnetwork is finally fine-tuned into the output pruned network.

3 Efficient Compact Network Learning

3.1 Overview

Differently from traditional network pruning, our method does not rely on a pre-trained and over-parameterized network, but directly searches and learns a compact network from scratch. As shown in Figure 1, our approach tackles the problem of joint search-and-training in a coarse to fine manner by alternating between two stages: **search subnetwork** and **update** (as shown by the dashed and solid arrows in Figure 2). Specifically, we first look for compact subnetworks from the target network and then update the trained subnetworks by mapping them to the target network. The best performing subnetworks are further fine-tuned by knowledge distillation as the final output. Inspired by Monte Carlo sampling and reinforcement learning, we count on random perturbations and iterative thresholding to efficiently search for a compact network during the search stage.

At the starting point, we first need to train the target network *coarsely* before searching. Similarly to previous works [51], [52], [53], [54], we introduce the concept of setting a flexible threshold for filter pruning, which requires that the weight indicate the importance of the corresponding filter. Along this line of reasoning, we have added a regularization term on the importance weights, in addition to the cross-entropy classification loss. Assuming that the target network $\mathcal{T}$ is a convolutional neural network with L convolutional layers at initialization, the complete training loss function ℓ_{coarse} to coarsely train the target network $\mathcal{T}$ is given by:

$$\ell_{coarse} = \ell_{ce}(y, z) + \lambda \left\| \mathbf{A} \right\|_2 . \tag{1}$$

The first term in the above equation is the cross-entropy loss between y and z, where $z = \mathcal{T}(x, \mathbf{\Omega}, \mathbf{A})$ represents the output vector of the target network $\mathcal{T}$. The symbol $\mathbf{\Omega} = \{\mathbf{W}_1, \mathbf{W}_2, \cdots, \mathbf{W}_L\}$ in the triplet refers to the parameters in the L convolutional layers of $\mathcal{T}$ and (x, y) denote the input data and the corresponding output labels, respectively. We have used $\mathbf{A} = [\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \cdots, \boldsymbol{\alpha}_L]$ to denote the importance weights associated with the convolution layers (this will be described in detail later in Sec. 3.2). The operator $\|\cdot\|_2$ refers to the ℓ_2 norm regularization of the weights, and λ is the hyperparameter.

After coarsely training the target network, we continue to iteratively search and obtain a series of compact networks $\mathcal{S}^0, \mathcal{S}^1, \ldots,$ as shown in Figure 2. These networks will be mapped back to the target network after proper training, which can be interpreted as pruning-based NAS in a coarse to fine manner. After the iterative search, we choose a compact network that performs the best on the validation set and fine-tune it until it reaches convergence. At the core of each search stage is a thresholding network, called **ThreshNet** in this work, which plays the role of adapting the thresholds of the input layers for network pruning.

3.2 Adaptive Searching for Compact Sub-network

For a given network $\mathcal{T}$, we denote the convolution filters in the l-th layer by $\mathbf{W}_l = [\boldsymbol{w}_{l,1}, \boldsymbol{w}_{l,2}, \cdots, \boldsymbol{w}_{l,C_l}] \in \mathcal{R}^{C_l \times (k^2 C_{l-1})}$, where C_l indicates the number of channels of the l-th layer and k represents the size of the kernel. For a given input feature map $\mathbb{I}_l$, the output feature maps of the l-th layer $\mathbb{O}_l$ can be expressed as:

$$\mathbb{O}_l = [\mathbb{I}_l * \boldsymbol{w}_{l,1}, \mathbb{I}_l * \boldsymbol{w}_{l,2}, \cdots, \mathbb{I}_l * \boldsymbol{w}_{l,C_l}], \tag{2}$$

where $*$ indicates the convolution operation and $\boldsymbol{w}_{l,i}$ represents the i-th convolutional filter in the l-th layer, corresponding to the i-th channel in the output feature maps $\mathbb{O}_l$. The goal of network pruning is to eliminate irrelevant filters to reduce the number of parameters and calculations in the target network. To this end, an intuitively simple idea is to learn a set of filter-dependent weights $\boldsymbol{\alpha}_l = [\alpha_{l,1}, \alpha_{l,2}, \cdots, \alpha_{l,C_l}]$ to directly characterize the importance of each filter. Thus, Eq. (2) can be rewritten as:

$$\begin{aligned} \mathbb{O} &= [\mathbb{I} * \boldsymbol{w}_{l,1} \cdot \tilde{\alpha}_{l,1}, \mathbb{I} * \boldsymbol{w}_{l,2} \cdot \tilde{\alpha}_{l,2}, \cdots, \mathbb{I} * \boldsymbol{w}_{l,C_l} \cdot \tilde{\alpha}_{l,C_l}] \\ st. \quad \tilde{\alpha}_{l,i} &= \frac{exp(\alpha_{l,i})}{\sum_{j=1}^{C_l} exp(\alpha_{l,j})}, \end{aligned} \tag{3}$$

where $\alpha_{l,i}$ represents the importance of the i-th output channel, the corresponding probability $\tilde{\alpha}_i$ can be calculated by the *softmax* function for all the importance weights of the same layer, and the sum of these probabilities should be one. Assuming $0 \approx \alpha_{l,i} \ll \alpha_{l,j}$ in the l-th layer, then according to Eq. (3), it can be inferred that the probability $\tilde{\alpha}_{l,i}$ is close to zero, implying that $\boldsymbol{w}_i \cdot \tilde{\alpha}_i$ is close to zero. Thus, the i-th filter can be pruned. In other words, the j-th filter should be preserved.

Algorithm 1: Adaptive filter selection for the l-th layer

Input: weights $\boldsymbol{\alpha}_l$ and filters $\mathbf{W}_l$

1 Add uniform noise on $\boldsymbol{\alpha}_l$ via Eq. (4) to get $\boldsymbol{p}_l$;

2 Calculate the probability $\tilde{\boldsymbol{p}}_l$ via $\tilde{p}_{l,i} = \frac{exp(p_{l,i})}{\sum_{j=1}^{C_l} exp(p_{l,j})}$;

3 Calculate the threshold for the l-th layer's weight: $\beta_l = \textbf{ThreshNet}(\boldsymbol{\alpha}_l)$;

4 Define $\mathcal{S}_p = \emptyset$ and $s = 0$;

5 **while** $s < \beta_l$ **do**

6 Find the largest element $\tilde{p}_{l,j_0}$ from $\tilde{\boldsymbol{p}}_l$, where $j_0 \notin \mathcal{S}_p$;

7 Update $\mathcal{S}_p = \mathcal{S}_p \bigcup \{j_0\}$;

8 Calculate the sum $s = s + \tilde{p}_{l,j_0}$;

9 Select the filters $\mathbf{W}_l^* = \{\boldsymbol{w}_{l,j}\}$, where $j \in \mathcal{S}_p$;

10 Select the filter-related weights $\boldsymbol{\alpha}_l^* = \{\alpha_{l,j}\}$, where $j \in \mathcal{S}_p$;

Output: $\boldsymbol{\alpha}_l^*$ and $\mathbf{W}_l^*$

Similarly to [55], we propose to improve the robustness by adding uniform noise to the filter-related weights $\boldsymbol{\alpha}_l$ as follows:

$$\boldsymbol{p}_l = \boldsymbol{\alpha}_l - \log(\log(\boldsymbol{u})), \qquad s.t. \quad \boldsymbol{u} \sim \mathcal{U}(0,1), \tag{4}$$

where $\mathcal{U}(0,1)$ is a uniform distribution on $[0,1]$. After that, with the weight $\boldsymbol{p}_l$, we can identify important filters by comparing the weights $\boldsymbol{p}_l$ with a threshold β_l. Specifically, we first compute the probability $\tilde{\boldsymbol{p}}_l$ by $\tilde{\boldsymbol{p}}_l = softmax(\boldsymbol{p}_l)$. Then, instead of directly selecting filters whose weights $\boldsymbol{p}_{l,i}$ are greater than β_l, we propose to select filters with the largest weights K_l such that the sum of these weights K_l is just greater than the threshold β_l. Our experimental results show that such filter selection strategy is more robust than simply selecting filters whose weights are larger than β_l. In the filter pruning process mentioned above, the key problem is how to determine the threshold β_l. Instead of selecting the threshold β_l by hand, which is difficult to choose appropriate thresholds for different layers, we propose to learn the thresholds β_l using a threshold learning network **ThreshNet**. The details of **ThreshNet** will be presented in the next subsection. The detailed pruning algorithm is summarized in **Algorithm 1**, where we denote the selected filters and the associated importance weights as $\mathbf{W}_l^*$ and $\boldsymbol{\alpha}_l^*$, respectively.

As shown in **Algorithm 1**, two optimization strategies are introduced into our search algorithm: 1) we propose to expand the search space by exploiting the randomness, i.e., filter-related weights are randomly perturbed by uniform noise before calculating the probabilities (lines 1 and 2 in **Algorithm 1**); 2) we propose to make the thresholds flexible with respect to each layer by leveraging the power of ThreshNet in Sec. 3.3 (line 3 in **Algorithm 1**). These two combined strategies can be interpreted as NAS based on Monte Carlo search [56] and reinforcement learning [35]. There are studies on randomness and pruning (e.g., [57], [58]) that demonstrate that random channel pruning is comparable to more complex pruning criteria. Meanwhile, NAS-based studies [59], [60], [61], [62] have illustrated that randomness brings about a complex search space. An important new insight of this study is that adding randomness in uniform noise sampling [55] can further increase robustness. We will illustrate the importance of randomness to robust pruning by comparing strong and weak randomness in Sec. 4.3.4.

3.3 Flexible Pruning via ThreshNet

Filter pruning operations are the soul of network pruning methods, because the filters kept at each layer will affect the accuracy of the pruned network. Therefore, the pruning operation must strike an optimal trade-off between performance (e.g., accuracy) and cost (e.g., pruning rate). Previous filter-wise pruning methods often manually control the number of pruned filters. Other filter pruning methods (e.g., [7]) search for the depth and width of the network and transfer knowledge from the trained large network to the searched small network. However, the optimality of filter pruning for a fixed number or pruning rate is questionable.

It is conceptually desirable to prune the filters more *adaptively* - e.g., via learned thresholding weights. We propose training a network called **ThreshNet** to automatically learn the threshold for each layer of the network. We formulate threshold learning as a sequential decision-making process, where an agent - ThreshNet parameterized by Φ - can make a sequence of decisions about threshold $\beta \in \mathcal{R}^L$ according to the filter weight α:

$$\boldsymbol{\beta} = \Phi(\mathbf{A}), \tag{5}$$

ThreshNet takes the importance weights $\mathbf{A} = [\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \cdots, \boldsymbol{\alpha}_L]$, the set of all convolutional layer's importance weights, as input and selects a threshold as shown in Figure 3) for each layer as the output decision. ThreshNet consists of two parts: The first part consists of several parallel and fully connected layers with different input sizes (e.g., green, blue, and purple blocks in the network in the upper right corner of Figure 3) corresponding to different sizes of α_l. These importance weights, after being transformed to the same dimension by the fully connected layers, will be concatenated and fed into the second part, which are several fully connected layers (corresponding to the orange blocks in Figure 3), responsible for extracting the features and selecting the appropriate thresholds for output. Note that the operation of selecting from a number of optional thresholds can be considered as an action in the decision-making process.

As the search process for compact networks continues, the forward propagation loss of each pruned subnetwork $\mathcal{S}$ is first calculated. The opposite of the average of losses in every K episode is then used as part of the reward (similar to the reward function in reinforcement learning [35]). To avoid finding too large subnets, the number of parameters is counted as a restriction, and its opposite is also used as a part of the reward as follows:

$$R = -\left(l_{avg} + \gamma \times param(\mathcal{S})\right), \tag{6}$$

where R refers to the reward, l_{avg} is the average loss, γ is the hyperparameter that balances the two parts of the reward, and $param(S)$ denotes the total number of parameters. We opt to train ThreshNet using a policy gradient algorithm to learn the policy π_Φ with parameters Φ by maximizing the rewards. Thus, the objective function is the expected reward for all sequences of actions of the agent β_l:

$$J(\Phi) = E_{\beta_l : L \sim \pi_\Phi}[R]. \tag{7}$$

Then we use the policy gradient algorithm approximated by [63] to optimize our ThreshNet as:

$$\nabla_\Phi J(\Phi) = \nabla_\Phi E_{\beta_i : l \sim \pi_\Phi}[R] \approx \frac{1}{K} \sum_{k=1}^{K} \sum_{l=1}^{L} R_k \nabla_\Phi \log \pi_\Phi(\beta_l | \boldsymbol{\alpha}_l), \tag{8}$$

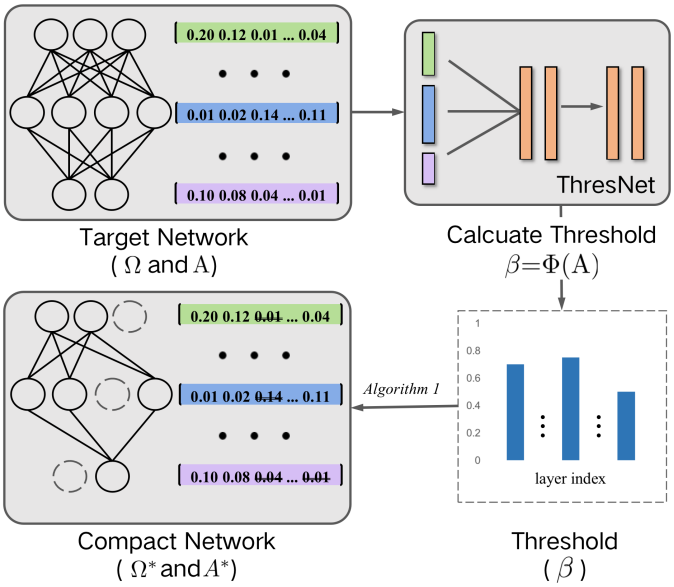


Fig. 3: The structure of our proposed ThreshNet. ThreshNet consists of fully connected layers; for input of network weight with different sizes, we process them with different fully connected layers. The final output is a pruning threshold for the corresponding input layer, which is used as a guide to retain the corresponding portion of the weights. Retained channels are determined along with weights.

where K is the number of episodes for a single gradient backward propagation, R_k is the reward computed in the k-th episode, and β_l is the threshold agent decided in layer l. Training ThreshNet makes it possible to adaptively learn the structure of compact subnets, giving us more flexibility in our pruning method. The sequence of thresholds learned by ThreshNet will be used to find a compact subnetwork $\mathcal{S}$, as we will elaborate next.

3.4 Knowledge distillation for robust pruning

Knowledge distillation is important to learn a robust pruning network. When the magnitude of pruning is large, pruning alone may lead to a poor generalization ability of the model. Such a lack of generalization properties or robustness is often hard to compensate for by training. In contrast, knowledge distillation can retain unpruned network information and features; more importantly, it can effectively alleviate the robustness problem. The key to knowledge distillation is a teacher-student network, i.e., by using a teacher network to guide the training of the student network, we can directly let the student network learn the generalization ability of the teacher network, which improves the robustness of pruning. A simple formulation of knowledge distillation using the teacher-student network [8] is as follows:

$$\begin{aligned}\ell_{soft}(z^*, z) &= -\sum_{i=1}^{N} p(\frac{z_i}{T}) log(p(\frac{z_i^*}{T})) \\ &= -\sum_{i=1}^{N} \frac{exp(\frac{z_i}{T})}{\sum_1^N exp(\frac{z_j}{T})} log(\frac{exp(\frac{z_i^*}{T})}{\sum_1^N exp(\frac{z_j^*}{T})}),\end{aligned} \quad (9)$$

where $z^* = \mathcal{S}(x, \mathbf{\Omega}^*, \mathbf{A}^*)$ denotes the output vector of the student network (i.e., the pruned network); $z = \mathcal{T}(x, \mathbf{\Omega}, \mathbf{A})$ corresponds to the output of the teacher network (i.e., the unpruned network). Note that T represents the temperature: As it converges to zero, the softmax output will converge to a one-hot vector, when the above equation constrains z^* and z to be as close as possible. Additionally, we add the cross-entropy loss as in Eq. (1) to encourage the pruned network to accurately predict the true label. Then the final equation of knowledge distillation boils down to the following.

$$\ell_{kd} = (1-\eta)\ell_{ce}((y, z^*)) + \eta\ell_{soft}(z, z^*). \quad (10)$$

Although distillation algorithms have been shown to be effective in improving the robustness of pruned networks [7], [45], [64], it is difficult to find sufficiently good teacher networks in our approach due to the omission of pre-training. As shown in Figure 2, the top row represents all the large networks in the algorithm, of which only the first network has been coarsely trained, while the subsequent large networks only update some parameters by "update", so it is not practical to use Eq. (10) directly in our algorithm. As a remedy, we propose the use of multiple teachers as labels to learn together to address this problem. Multi-teacher knowledge distillation methods have been used in knowledge distillation methods for a variety of problems, such as learning different levels of knowledge using different teacher combinations [65], further fine-tuning generic mini-models in the question answering domain using multi-teacher knowledge distillation methods [66] and refining compressed video action recognition models with multi-teacher knowledge [67]. However, this will be the first application of multi-teacher knowledge distillation to network pruning, to the best of our knowledge.

More specifically, multiple large networks are chosen together as teachers to improve the generalizability of the student network. Since large networks are not trained in our algorithmic process, choosing appropriate multiple-teacher networks is particularly important here. Our solution consists of a combination of good intuition and experience, i.e. we choose three large networks to jointly act as the teacher networks and assign different trade-off parameters. In summary, the target equation is constructed as follows:

$$\begin{aligned}\ell_{kd} &= (1-3\eta)\ell_{ce}(y, z^*) \\ &+ \eta(\ell_{soft}(z_1, z^*) + \ell_{soft}(z_2, z^*) + \ell_{soft}(z_3, z^*)),\end{aligned} \quad (11)$$

where z_1, z_2, z_3 represent the results of different teacher networks and η represent the tradeoff parameter. As for the choice of teacher networks, we choose the first, third, and fifth large networks because these networks have more global information and are not too similar. More details about the justification for our choice of three teacher networks can be found in Sec. 4.3.3.

3.5 Overall Algorithm

As discussed in Sec. 3.2, this paper uses the superscript '*' to denote the searched structure, i.e., the reconstructed compact network. Thus, for the reconstructed network, the output of each layer in Eq. (3) can be written as:

$$\begin{aligned}\mathbb{O} &= [\mathbb{I} * \mathbf{W}_1^* \cdot \tilde{\alpha}_1^*, \mathbb{I} * \mathbf{W}_2^* \cdot \tilde{\alpha}_2^*, \cdots, \mathbb{I} * \mathbf{W}_{C_l}^* \cdot \tilde{\alpha}_{K_l}^*] \\ s.t. \quad \tilde{\alpha}_i^* &= \frac{exp(\alpha_i^*)}{\sum_{j=1}^{K_l} exp(\alpha_j^*)},\end{aligned} \quad (12)$$

where $[\alpha_1^*, \alpha_2^*, \cdots, \alpha_{K_l}^*] = \boldsymbol{\alpha}_l^*$ is the output of **Algorithm 1** and in which K_l represents the number of preserved channels in the l-th layer. By sampling each layer of the network, a set of $[\mathbf{W}_1^*, \mathbf{W}_2^*, \cdots, \mathbf{W}_L^*] = \mathbf{\Omega}^*$ and $[\boldsymbol{\alpha}_1^*, \boldsymbol{\alpha}_2^*, \cdots, \boldsymbol{\alpha}_L^*] = \mathbf{A}^*$ can be obtained, respectively. The sampled network can be considered a compact subnetwork $\mathcal{S}$, whose parameters and weights can be

Algorithm 2: Overall Search-and-Training Algorithm

Input: The whole available training set $\mathcal{D}$
Initialize the target network $\mathcal{T}^0$ with filter-related weight $\boldsymbol{\alpha}$ using Eq (3);
Split the whole training set $\mathcal{D}$ into $\mathcal{D}_{train}$ and $\mathcal{D}_{valid}$;
Get coarsely trained network $\mathcal{T}^1$ on $\mathcal{D}$ by Eq (1);
for $u = 1 \rightarrow U$ **do**
  Sample a sub-network $\mathcal{S}^u(\mathbf{\Omega}^*, \mathbf{A}^*) \subseteq \mathcal{T}^u$ based on **Algorithm 1**;
  for $m = 1 \rightarrow M$ **do**
    Sample train batch $\mathcal{B}_t = (x, y)$ from $\mathcal{D}_{train}$
    Optimize $\mathbf{\Omega}^*$ on the $\mathcal{B}_t$ by Eq (13)
  for $n = 1 \rightarrow N$ **do**
    Sample valid batch $\mathcal{B}_v == (x, y)$ from $\mathcal{D}_{valid}$
    Optimize $\mathbf{A}^*$ on the $\mathcal{B}_v$ by Eq (14)
  Update $\mathcal{T}^u$ via optimized $\mathbf{\Omega}^*$ and $\mathbf{A}^*$ to get $\mathcal{T}^{u+1}$;
if *w/o Distill* **then**
  Select the best performing network $\mathcal{S}$ among $\{\mathcal{S}^1, \mathcal{S}^2, \cdots, \mathcal{S}^U\}$;
  Fine-tune it without distillation on $\mathcal{D}$ via Eq 1;
if *w Distill* **then**
  Sample P sub-networks in $\mathcal{T}^U$ and select the best performing network $\mathcal{S}$;
  Fine-tune it with distillation on $\mathcal{D}$ as described in Section 3.4;
Output: Pruned network $\mathcal{S}$

denoted by $\mathbf{\Omega}^*$ and $\mathbf{A}^*$, respectively. By feeding this information back to the sampler through the updater, as shown in Figure 1, we observe that the search for the most compact subnetwork can be solved iteratively in a coarse-to-fine manner.

Just like the NAS method [35], we can update the parameters and weights on the training set $\mathcal{D}_{train}$ and the validation sets $\mathcal{D}_{valid}$ successively, which can be formulated as

$$\mathbf{\Omega}^* = \underset{\mathbf{\Omega}^*}{\operatorname{argmin}} \sum^{\mathcal{B}_t} \ell_{train}(y, \mathcal{S}(x, \mathbf{\Omega}^*, \mathbf{A}^*)), \tag{13}$$

$$\mathbf{A}^* = \underset{\mathbf{A}^*}{\operatorname{argmin}} \sum^{\mathcal{B}_v} \ell_{val}(y, \mathcal{S}(x, \mathbf{\Omega}^*, \mathbf{A}^*)), \tag{14}$$

where (x, y) indicates the input and the corresponding label, ℓ_{train} and ℓ_{val} are the loss of the network cross-entropy classification, $\mathcal{B}_t$ denotes the batches sampled from $\mathcal{D}_{train}$, and so is $\mathcal{B}_v$ from $\mathcal{D}_{valid}$. After obtaining the optimized $\mathbf{\Omega}^*$ and $\mathbf{A}^*$, we map these parameters back to the target network, that is, we put the trained parameters back to their original locations. In this way, the target network $\mathcal{T}$ can still be optimized and updated despite the fact that it has not been trained again after the initial coarse training. To distinguish the optimized target network from the initial one, we label the subsequently updated network as $\mathcal{T}^u$ and the corresponding compact subnet as $\mathcal{S}^u$, where $u \in (0, U]$.

Intuitively, the sampling of compact subnets can benefit from the experience of the previous search in the next search procedure. In other words, the learned knowledge in the NAS space can be iteratively fed back to the sampler, which could further improve search efficiency and training performance. Finally, we can obtain various versions of the pruned networks depending on the different requirements for speed and accuracy: for the faster version, we might choose the best one among the subnetworks and fine-tune it to convergence; for the more robust version, we might randomly sample several subnetworks in the larger network and distill the best one among them. In summary, the complete procedure of our joint search and training approach is referred to as **Algorithm 2**.

γ	0	0.5	1.0	1.5	2.0	2.5
Pruning Rate	2%	16%	31%	44%	53%	61%

TABLE 1: Pruning rate for different gamma. The pruned network is the ResNet-20 model based on the CIFAR-10 dataset.

4 Experimental Results

4.1 Experimental Setting

Datasets. We validate our network pruning method on several main datasets. Among them, CIFAR-10 contains 60,000 images classified into 10 classes. The training set has 5,000 images per class and 50,000 images in total. The test set contains 1,000 images per class and 10,000 images in total. CIFAR-100, similar to CIFAR-10, contains 50,000 training images and 10,000 test images, categorized into 100 instead of 10 classes. All CIFAR images are 32×32 color images. In contrast, ImageNet is a large-scale image classification dataset that contains 1,000 classes, 1.28 million training images, and 50,000 validation images. Each image was cropped to the size of 224×224 during the preprocessing step. Therefore, the memory cost in our experiments has been less of a concern due to the relatively small size of these images.

Implementation Details. For searching and training, we randomly extract 80% of the official training images as the training set $\mathcal{D}_{train}$ in **Algorithm 2**, and the rest as the validation set $\mathcal{D}_{val}$. In our implementation, we search for compact networks with thresholds in the range of $[0.6, 0.65, 0.7, 0.75, 0.8]$ for each layer. The following hyperparameter is used in our experiment: $\lambda = 0.1$, $\gamma = 2.0$, $P = 200$, $U = 20$, $\eta = 0.2$, and update ThreshNet for $K = 20$ episodes after each sampling. All these parameters and weights will be initialized by the Kaiming normal in PyTorch. And we set $\gamma = 2$ to achieve a compression rate of about 50% shown later, where γ is a hyperparameter that controls the pruning ratio: The larger γ, the higher the pruning rate. Since γ has no strong constraint on FLOPs, the compression rate obtained is also approximate. For comparison, as shown in Table 1, we show the change of pruning rate brought by different values of γ.

For different datasets, we have varied their parameter settings as follows:

1) On CIFAR dataset, we use stochastic gradient descent (SGD) with a momentum of 0.9 and a weight decay of 0.00005 as optimizer. At the beginning, we train the target network coarsely for 100 epochs, with a batch size of 128. The learning rate starts from 0.1 and is reduced by a cosine scheduler. Then we search for a compact network with $U = 30$, whose parameters are optimized in epochs of $\mathcal{D}_{train}$ for M ($M = 40$ for $t \leqslant 20$, 30 for $t > 20$) with learning rates of 0.05/0.01, corresponding to different values of M. And the weights are optimized on $\mathcal{D}_{val}$ for $N = 5$ epochs with a fixed learning rate of 0.001. In the fine-tuning stage, we set a batch size of 256, a learning rate of 0.01, and optimize the selected compact network until convergence. The number of GPUs we used is consistent with the compared method: for the experiments on the CIFAR-10 dataset, we use one NVIDIA Titan XP GPU for training and searching, and one NVIDIA 1080Ti for CIFAR-100.

Network	Method	Pre-trained	Fine-tuned	FLOPs/Pruned	Acc/Drop	Time Cost
ResNet20	SFP [22]	✓	✗	2.43e7/42.2%	90.83/1.37↓	5h10m
	FPGM [20]	✓	✗	2.43e7/42.2%	91.09/1.11↓	7h20m
	TAS [7]	✓	✓	2.24e7/45.0%	92.88/0.00↓	8h40m
	Hinge [17]	✓	✓	2.22e7/45.0%	91.84/0.70↓	-
	Ours (w/o Distill)	✗	✓	2.27e7/45.1%	91.98/0.84↓	3h50m
	Ours	✗	✓	2.21e7/46.6%	92.05/0.77↓	5h07m
ResNet56	SFP [22]	✓	✗	5.94e7/52.6%	93.35/0.24↓	4h10m
	FPGM [20]	✓	✗	5.94e7/52.6%	93.49/0.32↓	9h50m
	TAS [7]	✓	✓	5.95e7/52.7%	93.69/0.77↓	18h10m
	Hinge [17]	✓	✓	6.27e7/50.0%	93.69/0.74↓	-
	HRank [64]	✓	✓	6.27e7/50.0%	93.17/0.09↓	-
	ResRep [27]	✓	✗	5.89e7/52.9%	93.71/0.00↓	-
	Adapt-DCP [18]	✓	✓	5.67e7/54.5%	93.77/-0.03↓	-
	EagleEye [24]	✓	✗	6.23e7/50.4%	94.66/-1.40↓	-
	Ours (w/o Distill)	✗	✓	6.11e7/50.9%	93.71/0.46↓	4h50m
	Ours	✗	✓	6.11e7/50.9%	94.11/0.06↓	7h00m
Resnet110	SFP [22]	✓	✗	1.50e8/40.8%	93.86/0.18↓	15h40m
	FPGM [20]	✓	✗	1.21e8/52.3%	93.85/0.17↓	10h30m
	TAS [7]	✓	✓	1.19e8/53.0%	94.33/0.64↓	21h
	HRank [64]	✓	✓	1.49e8/41.2%	94.23/0.73↓	-
	Ours (w/o Distill)	✗	✓	1.08e8/58.0%	94.32/0.60↓	7h31m
	Ours	✗	✓	1.06e8/59.0%	94.81/0.11↓	10h20m
Resnet164	TAS [7]	✓	✓	1.78e8/28.1%	94.00/1.47↓	54h45m
	Hinge [17]	✓	✓	1.39e8/46.3%	94.60/0.43↓	-
	Ours (w/o Distill)	✗	✓	1.12e8/54.4%	94.22/0.93↓	14h28m
	Ours	✗	✓	1.12e8/54.4%	94.41/0.74↓	18h50m

TABLE 2: Comparison of different pruning algorithms for ResNet on CIFAR10. 'FLOPS / pruned' means the calculation and pruning rate. 'Acc / drop' means accuracy and performance drop. The "✓" and "✗" under "Pre-trained" and "Fine-tuned" indicate whether the corresponding method needs to be pretrained before pruning or optimized afterward, respectively. Note that the data of SFP and FPGM are from the training log published by the authors, and the data of TAS are from the open source code of the author.

2) On the ImageNet dataset, we optimize the parameters via Adam with a weight decay of 0.00001 and the weights via the same SGD as CIFAR. For the ResNet50 network, we coarsely train 40 epochs with an initial learning rate of 0.1 and search 20 compact networks. M is set to be 10 with a learning rate of 0.001 and N is set to be 2. For MobileNet V2, we coarsely train 60 epochs and keep the other settings constant. When fine-tuning, we set the initial learning rate to 0.001 and divide it by 10 every 20 epochs. All experiments in the ImageNet dataset use four NVIDIA Titan XP GPUs with a batch size of 256. In all comparison experiments, we provide two different sets of experimental controls: *Ours (w/o Distill)* and *Ours*, where *Ours (w/o Distill)* means that no distillation is added and moderate fine-tuning is performed in this group, and correspondingly, *Ours* means that distillation is done by fine-tuning until it reaches convergence.

4.2 Comparison Against Other Competing Methods

On CIFAR-10 Dataset. We evaluate our method on ResNet-20/56/110/164. We have compared several recently proposed pruning methods with our method, including the soft filter pruning (SFP) [22] method, the geometric median-based filter pruning (FPGM) [20] method, the Transformable Architecture Search (TAS) [7] method, the joint pruning method based on group sparsity (Hinge) [17] and (HRank) [64] method for pruning based on the rank of a matrix. Fairness to them, the results of other methods were borrowed directly from their papers [17], [64]. Regarding the consumed time, we calculate the sum of the pre-training time, pruning, and tuning time, where the pruning time is not reported in the article, so we obtain it through the code and hyperparameters provided by [7], [20], [22].

In Table 2, we show the comparisons of FLOP, accuracy, and time cost in CIFAR-10. With fewer or close FLOPs, *Ours (w/o Distill)* can achieve nearly $2\times$ speed increase and even $0.3 \sim 1.0\%$ higher accuracy, compared to [7], [20], [22], [70]. Compared to the best performance method [7], our algorithm performs slightly worse on ResNet20 and 56, has lower accuracy on ResNet110 with lower flops, and has better performance on ResNet164. Although our algorithm slightly falls behind [7] in terms of accuracy on some networks, it is $3 \sim 5\times$ faster in

Network	Method	FLOPs/Droped	Acc/Droped	Time
ResNet20	SFP [22]	24.3M/42.2%	64.37/3.25↓	-
	FPGM [20]	24.3M/42.2%	66.86/0.86↓	-
	TAS [7]	22.4M/45.0%	68.90/-0.21↓	8.3h
	NA [44]	22.4M/45.0%	70.03/-0.51↓	-
	Ours (w/o ThreshNet)	20.5M/49.7%	67.28/1.64↓	5.5h
	Ours (w/o Distill)	22.7M/45.9%	66.48/2.44↓	5.0h
	Ours	22.2M/46.2%	68.77/0.15↓	7.5h
ResNet56	SFP [22]	59.4M/52.6%	68.79/2.61↓	-
	FPGM [20]	59.4M/52.6%	69.66/1.75↓	-
	TAS [7]	61.2M/51.3%	72.25/0.93↓	20.0h
	Ours (w/o ThreshNet)	67.2M/51.1%	70.07/2.86↓	14.3h
	Ours (w/o Distill)	67.2M/51.1%	70.63/2.26↓	11.0h
	Ours	59.8M/56.5%	71.15/1.74↓	16.9h
Resnet110	SFP [22]	121M/52.3%	71.28/2.86↓	-
	FPGM [20]	121M/52.6%	72.55/1.59↓	-
	TAS [7]	120M/51.3%	73.16/1.90↓	34.5h
	NA [44]	120M/51.3%	75.96/-0.28↓	-
	Ours (w/o ThreshNet)	108M/58.0%	71.69/2.73↓	13.0h
	Ours (w/o Distill)	108M/58.0%	72.26/2.16↓	9.5h
	Ours	108M/58.0%	73.12/1.30↓	15.5h
VGG-16	GCP [68]	382M/39.1%	72.01/1.18↓	-
	SSS [69]	308M/50.6%	72.95/0.24↓	-
	Ours	322M/48.4%	74.63/1.12↓	4.5h

TABLE 3: Comparison of different pruning algorithms for ResNet on CIFAR-100. The parameters in the above table are consistent with Table2, where "Ours (w/o ThreshNet)" means pruning with a fixed pruning rate for each layer in the network.

Method	FLOPs/Pruned	Acc/Droped Top-1	Acc/Droped Top-5	Time
SFP [22]	2.38e9 / 41.8%	74.61 / 1.54↓	92.06 / 0.81↓	130h
FPGM [20]	2.36e9 / 42.2%	75.50 / 0.65↓	92.63 / 0.21↓	120h
MetaPruning [23]	2.00e9 / 51.3%	75.40 / 1.20↓	-	-
TAS [7]	2.31e9 / 43.5%	76.20 / 1.26↓	93.07 / 0.48↓	110h
HRank [64]	2.30e9 / 43.8%	74.98 / 1.17↓	92.33 / 0.54↓	-
Hinge [17]	2.30e9 / 46.6%	74.70 / -	-	-
DMCP [45]	2.20e9 / 46.1%	76.20 / 0.40↓	-	168h
LeGR [71]	2.37e9 / 42.0%	75.30 / 0.40↓	92.40 / 0.20↓	-
ResRep [27]	1.88e9 / 54.5%	76.15 / 0.00↓	92.89 / -0.02↓	-
Adapt-DCP [18]	1.96e9 / 52.4%	75.15 / 0.86↓	92.30 / 0.63↓	-
EagleEye [24]	2.00e9 / 51.0%	76.40 / -	92.89 / -	-
Ours (w/o Distill)	2.25e9 / 44.9%	75.77 / 0.41↓	92.55 / 0.76↓	92h
Ours	2.25e9 / 44.9%	76.04 / 0.14↓	92.83 / 0.48↓	121h

TABLE 4: Comparison of different pruning algorithms of ResNet50 on ImageNet.

Method	FLOPs/Pruned	Acc/Droped Top-1	Acc/Droped Top-5	Time
AMC [49]	1.50e8 / 50.0%	70.80 / 1.00↓	-	-
TAS [7]	1.49e8 / 50.3%	70.90 / 1.18↓	89.74 / 0.76↓	103h
MetaPruning [23]	1.40e8 / 53.4%	68.20 / 4.50↓	-	
DMCP [45]	2.11e8 / 29.3%	72.40 / 0.10↑	-	132h
	0.97e7 / 67.7%	67.00 / 2.40↓	-	116h
LeGR [71]	2.20e8 / 31.0%	71.60 / 0.30↓	-	-
ManiDP [25]	1.92e8 / 37.2%	71.42 / 0.38↓	90.28 / 0.15↓	-
Adapt-DCP [18]	2.09e8 / 30.7%	71.39 / 0.49↓	90.09 / 0.21↓	-
Ours (w/o Distill)	1.49e8 / 50.3%	70.92 / 1.10↓	90.10 / 1.23↓	73h
Ours	1.50e8 / 50.2%	71.25 / 0.77↓	90.67 / 0.66↓	92h
Ours (w/o Distill)	2.05e8 / 31.5%	71.13 / 0.89↓	90.33 / 1.00↓	73h
Ours	2.12e8 / 29.4%	71.61 / 0.41↓	91.09 / 0.24↓	92h

TABLE 5: Comparison of different pruning algorithms of MobileNet V2 on ImageNet.

speed. At the same time, we note that our baseline performance is lower than that of [7], which is caused by hardware issues, lack of special training skills, and lack of experience in parameter tuning. By contrast, our result for ResNet56 is highly competent against those reported in EagleEye [24] (e.g., with comparable FLOPs and top-1 Acc) A more intuitive comparison is "Dropped Acc", in which we are closer to [7]. Note that only [7] uses the setting of $batchsize = 256$ (set to 128 in other implementations), implying more GPU resources. We believe that the search for network depth and width at the same time is the reason for longer running times and higher recognition accuracy.

On CIFAR-100 Dataset. In Table 3, we compare the performance of different algorithms on ResNet20, 56, 110, and VGG-16. And a network adjustment method(NA) [44] has been introduced as a comparison. Because the settings of the experimental parameters on CIFAR-100 are not provided in [20], [22], we cannot compare the running time for the sake of fairness. To prune the ResNet network, our method can maintain the efficiency of other methods, and the performance is close to [20], [22], and only slightly worse than [7], but with a certain difference than [44]. This is because on the CIFAR-100 dataset, there are only 600 training pictures of each category and only 480 pictures per category in $\mathcal{D}_{train}$, which causes our method to encounter serious underfitting problems while learning the compact network from scratch. Comparatively, a network tuning or search method similar to [44] yields a model with better topology and therefore better performance for the same data size. For VGGNet, our method still outperforms the sparsity-based structure selection method (SSS) [69] and the genetic algorithm-based channel pruning method (GCP) [68].

On ImageNet Dataset. In Table 4 and Table 5, We have validated our method on ResNet-50 and MobileNet V2 networks using four NVIDIA 2080Ti GPUs . For a more comprehensive comparison, we have added a meta-learning-based method (metapruning) [23], a Markov chain-based pruning method (DMCP) [45], (LeGR) [71]that use global rank for model compression, and a dynamic pruning method (ManiDP) [25]. Compared to those state-of-the-art algorithms, the proposed method still shows competitive results. For example, we can prune ResNet-50 in 90 hours, and the pruned network can achieve $75.77/92.55\%$ accuracy, an accuracy that exceeds most comparison methods and is only slightly worse than DMCP, ResRep and EagleEye, but much faster. This is because in the training phase of DMCP and TAS, it is necessary to sample many sub-network structures and perform forward/reverse calculations, which requires a lot of computing resources and long-term training. And after the training phase, a retraining phase is required, and this part also requires a lot of computation. While ResRep and EagleEye introduced additional parameters for evaluation during the training phase, which also increased the complexity of the algorithm. Compared with these methods, due to the pursuit of efficiency, our algorithm does not

have a large number of sampling sub-networks, so the accuracy is slightly lower than DMCP in some experiments.

Especially in the time complexity of the algorithm, please note that the training time of the entire ResNet50 network is close to 80 hours, which means that we can complete the entire search algorithm while other algorithms are training the unpruned network. In our implementation, the running time increases to 121 h (due to the addition of random sampling and distillation), but is still more efficient than DMCP and TAS with comparable accuracy/FLOPs. We have also introduced the lightweight MobileNet V2 network ($1.0\times$) as the pruning target for comparison. Because different accuracies/FLOPs are achieved by competing methods, we show more comprehensive experiments for better illustration. As can be seen in Table 4 and Table 5, *Ours (w/o Distill)* version achieves more accuracy than the others, using just over half the time. For the robuster version, performance is further exceeded while maintaining the advantage of time complexity. It should be noted that DMCP achieves an increase in the precision of the pruning model at 211M FLOPs, which we believe is due to the use of random sampling.

4.3 Ablation Study

4.3.1 The effect of flexible pruning

To show the impact of flexible pruning, we have implemented a baseline version of the proposed method - i.e., pruning with a manually determined fixed pruning rate(the *w/o ThreshNet* version in Table 3). We calculate the appropriate pruning rate based on the implementation of flexible pruning and apply it to each layer of the network. We validated ResNet-20, 56, and 110 in the CIFAR-100 data set and ensured that the control experiments had the same number of FLOPs. Compared to a fixed pruning rate, the flexible pruning method can effectively speed up the search and improve the performance of the resulting network. For example, when resnet56 was cultivated, our flexible pruning method achieved a performance improvement 0.57% in less time than the fixed pruning method. Note that we are just training a simple fully connected network as a ThreshNet with a basic reinforcement learning strategy, so the performance of our algorithm can also be improved by using more powerful networks and more effective reinforcement learning methods.

4.3.2 Selecting the optimal subnetwork

An essential part of our approach is to select the best performing subnets to fine-tune at the end of the training and search process. To better illustrate the search-and-training procedure, we provide the performance of all sampled networks. After the search-and-training procedure (corresponding to lines 4-12 in the **Algorithm 2**), we can get some good but not well-trained subnets. As described in **Algorithm 2**, line 13, we select and optimize the network with the best performance. In our method, it is unnecessary to ensure that every subnetwork is well trained after search-and-training procedure due to the following two reasons: first, training every sampled subnetwork to convergence will greatly improve the time cost and calculation cost of the algorithm; second, in the process of searching, a well-trained subnetwork will lead to the reduction of search space, which limits the subnetwork to be searched later. Therefore, bias about the last sampled network arising from iterative training also does not occur. In our experiments, the best performing subnet usually appears in the middle of the training epoch, as shown in Figure 4.

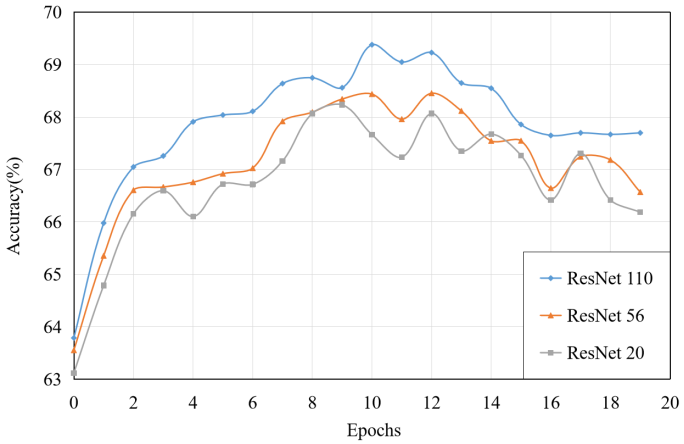


Fig. 4: We show the performance of all sampled subnetworks during the search of resnet-110 on Cifar100 dataset. Before the search, the target network has been coarsely trained; after the search, the best performing subnetwork will be fine-tuned to convergence.

4.3.3 The impact of distillation teachers

Since there is no well-trained large network in our new proposed paradigm for pruning networks from scratch, choosing the most appropriate teacher network for knowledge distillation learning will be the key to improving the robustness of pruning networks. Most other pruning methods use the unpruned network as the teacher network, and the closest thing to the unpruned network in our framework is the roughly trained network corresponding to the top left of Figure 1, but can this be the teacher network or do we need other large networks together as the teacher network? To answer this question, we validate the single teacher network and the multigroup multi-teacher network, as shown in Figure 5a.

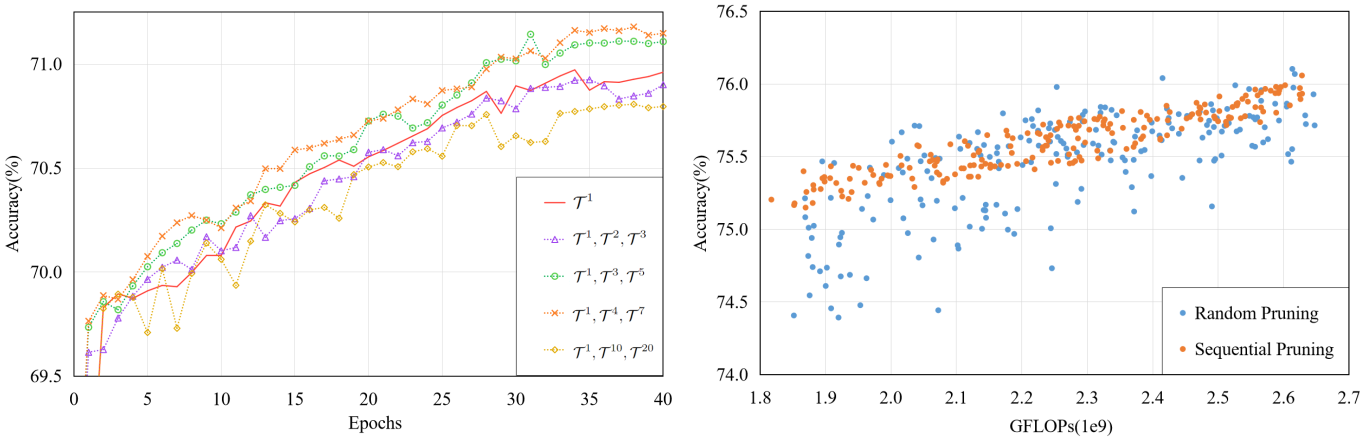
Note that our experiments were validated on CIFAR100 with ResNet56, where $\eta = 0.2$ was set for Eq.11. For comparative observations, the solid red line in Figure 5a represents the selection of $\mathcal{T}^1$ as the teacher alone; and the dashed line represents the use of three large networks as the teacher network. It can be seen from the figure that the effect of using the first large network alone and using three consecutive large networks as teacher networks is close, which is due to the fact that these three networks are quite close in terms of parameter distribution; while another set of noteworthy contrasts is that when we use the networks that are later in the search sequence (yellow curves in the figure) as teacher networks instead, it leads to a decrease in distillation effect (0.32%), which is probably due to the fact that these large networks contain too much redundant information. Therefore, we have chosen a compromised solution($\mathcal{T}^1$, $\mathcal{T}^4$ and $\mathcal{T}^7$) in the experiments.

4.3.4 Impact of random selection

In this section, we will verify the validity of randomness in our pruning strategy. To balance efficiency and randomness, we introduce strong randomness and weak randomness, corresponding to "random" and "Gumbel-Softmax" in the table, respectively. We also explore the application of randomness in the training process and the final fine-tuning process. As shown in Table 6, where our experiments are based on CIFAR10 and ResNet56, we can make the following observations:

Network	Random sample on Searching Stage	Random sample before Fine-tuning Stage	FLOPs	Accuracy	Time Cost
ResNet56	random	random	6.08e7	93.97	50±5h
	✗	✗	6.10e7	93.48	**4±0.3h**
	gumbel-softmax	random	6.10e7	**94.13**	18±0.7h
	gumbel-softmax	✗	6.11e7	93.71	**4±0.5h**
	gumbel-softmax	gumbel-softmax	6.11e7	**94.11**	8±0.8h

TABLE 6: Pruning ResNet-56 at a pruning rate of about 50% on CIFAR-10. "random" represents a random sample, and "Gumbel-Softmax" denotes the increase of randomness using Gumbel-Softmax and ✗ means that there is no randomness and the channel is manually pruned sequentially according to its corresponding weight α from the largest to the smallest.



a) Results of different teacher networks

b) Results of different sampling modes

Fig. 5: Ablation studies on distillation teachers and random selection. a) The impact of choosing different large networks as teacher networks on knowledge distillation; b) Accuracy/FLOPs of the 200 sampled subnets on ResNet-50.

1) Compared with the one without randomness, whether it is adding weak randomness or strong randomness, our method can be well combined to improve the performance of the pruning network;
2) Strong random may bring performance improvement, but it will also significantly increase the time complexity of the algorithm;
3) Random sampling before the Fine-tuning Stage is the main source of performance improvement;
4) Using Gumbel-Softmax in the training phase is not only in terms of time complexity, but also better than strong randomness in performance.

Based on these observations, we believe that when searching for the best filter combination, using random sampling can obtain a better model. Compared to completely random sampling, Gumbel-Softmax is slightly worse in performance, but more friendly in terms of time complexity. Therefore, we have added Gumbel-Softmax-based sampling to the pruning algorithm to achieve better performance at a limited time cost. As shown in Figure 5b, compared to the traditional sequence pruning method, random pruning has a significant improvement in optimal results: By repeating the sampling 200 times on ResNet-50 and keeping the pruning rate floating between 35% and 55%, it can be seen that traditional sequence pruning is more stable and balanced, but random pruning can achieve the best accuracy results.

5 Conclusion

In this article, we propose a new joint search and training method designed to learn a compact network directly from scratch. First, we propose to iteratively sample and update compact networks to approximate the target network. Unlike most previous methods, we do not follow the rule of pruning after training, but jointly search and train for learning compact networks directly from unclipped networks. Second, we have developed a flexible pruning method that flexibly adjusts the weight and size of each layer by learning a set of filter-related weights. Finally, we propose to optimize ThreshNet by reinforcement learning and knowledge distillation. In standard benchmark data sets and network architecture, our proposed method achieves state-of-the-art efficiency improvement with highly competitive classification accuracy. Future work includes further extension of the proposed adaptive search and training strategy by fast subnet evaluation [24] and considering the FLOP utilization ratio [44].

Acknowledgments

This work was supported in part by the Natural Science Foundation of China under Grant 61991451 and Grant 61836008. Xin Li's work is partially supported by the NSF under grant IIS-2114664, CMMI-2146076, and the WV Higher Education Policy Commission Grant (HEPC.dsr.23.7).

REFERENCES

[1] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," *arXiv preprint arXiv:1810.05270*, 2018.

[2] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *arXiv preprint arXiv:1808.05377*, 2018.

[3] X. Lu, H. Huang, W. Dong, X. Li, and G. Shi, "Beyond network pruning: a joint search-and-training approach," in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, C. Bessiere, Ed. International Joint Conferences on Artificial Intelligence Organization, 7 2020, pp. 2583–2590, main track.

[4] X. Dong and Y. Yang, "Searching for a robust neural architecture in four gpu hours," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 1761–1770.

[5] H. Liu, K. Simonyan, and Y. Yang, "Darts: Differentiable architecture search," *arXiv preprint arXiv:1806.09055*, 2018.

[6] S. Zhang, X. Zheng, C. Yang, Y. Li, Y. Wang, F. Chao, M. Wang, S. Li, J. Yang, and R. Ji, "You only compress once: Towards effective and elastic bert compression via exploit-explore stochastic nature gradient," *arXiv preprint arXiv:2106.02435*, 2021.

[7] X. Dong and Y. Yang, "Network pruning via transformable architecture search," in *Advances in Neural Information Processing Systems*, 2019, pp. 759–770.

[8] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[9] C. Liu, B. Zoph, M. Neumann, J. Shlens, W. Hua, L.-J. Li, L. Fei-Fei, A. Yuille, J. Huang, and K. Murphy, "Progressive neural architecture search," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 19–34.

[10] H. Zhou, J. M. Alvarez, and F. Porikli, "Less is more: Towards compact cnns," in *European Conference on Computer Vision*. Springer, 2016, pp. 662–677.

[11] E. L. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus, "Exploiting linear structure within convolutional networks for efficient evaluation," in *Advances in neural information processing systems*, 2014, pp. 1269–1277.

[12] J. M. Alvarez and M. Salzmann, "Learning the number of neurons in deep networks," in *Advances in Neural Information Processing Systems*, 2016, pp. 2270–2278.

[13] H. Kim, M. U. K. Khan, and C.-M. Kyung, "Efficient neural network compression," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 12 569–12 577.

[14] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," in *Advances in neural information processing systems*, 2016, pp. 2074–2082.

[15] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural network," in *NIPS*, 2015.

[16] F. Meng, H. Cheng, K. Li, H. Luo, X. Guo, G. Lu, and X. Sun, "Pruning filter in filter," in *NeurIPS*, 2020.

[17] Y. Li, S. Gu, C. Mayer, L. V. Gool, and R. Timofte, "Group sparsity: The hinge between filter pruning and decomposition for network compression," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 8018–8027.

[18] J. Liu, B. Zhuang, Z. Zhuang, Y. Guo, J. Huang, J. Zhu, and M. Tan, "Discrimination-aware network pruning for deep model compression," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.

[19] S. Gao, F. Huang, W. Cai, and H. Huang, "Network pruning via performance maximization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 9270–9280.

[20] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4340–4349.

[21] J. Li, Q. Qi, J. Wang, C. Ge, Y. Li, Z. Yue, and H. Sun, "Oicsr: Out-in-channel sparsity regularization for compact deep neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 7046–7055.

[22] Y. He, G. Kang, X. Dong, Y. Fu, and Y. Yang, "Soft filter pruning for accelerating deep convolutional neural networks," in *IJCAI International Joint Conference on Artificial Intelligence*, 2018.

[23] Z. Liu, H. Mu, X. Zhang, Z. Guo, X. Yang, K.-T. Cheng, and J. Sun, "Metapruning: Meta learning for automatic neural network channel pruning," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 3296–3305.

[24] B. Li, B. Wu, J. Su, and G. Wang, "Eagleeye: Fast sub-net evaluation for efficient neural network pruning," in *European conference on computer vision*. Springer, 2020, pp. 639–654.

[25] Y. Tang, Y. Wang, Y. Xu, Y. Deng, C. Xu, D. Tao, and C. Xu, "Manifold regularized dynamic network pruning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 5018–5028.

[26] Z. Wang, X. Xie, and G. Shi, "Rfpruning: A retraining-free pruning method for accelerating convolutional neural networks," *Applied Soft Computing*, vol. 113, p. 107860, 2021.

[27] X. Ding, T. Hao, J. Tan, J. Liu, J. Han, Y. Guo, and G. Ding, "Resrep: Lossless cnn pruning via decoupling remembering and forgetting," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 4510–4520.

[28] Z. Wang, C. Li, and X. Wang, "Convolutional neural network pruning with structural redundancy reduction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 14 913–14 922.

[29] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.

[30] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 6848–6856.

[31] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7132–7141.

[32] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "Ghostnet: More features from cheap operations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1580–1589.

[33] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proceedings of the aaai conference on artificial intelligence*, vol. 33, 2019, pp. 4780–4789.

[34] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," *arXiv preprint arXiv:1802.03268*, 2018.

[35] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8697–8710.

[36] X. Dong and Y. Yang, "One-shot neural architecture search via self-evaluated template network," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 3681–3690.

[37] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," *arXiv preprint arXiv:1812.00332*, 2018.

[38] L. Xie, X. Chen, K. Bi, L. Wei, Y. Xu, L. Wang, Z. Chen, A. Xiao, J. Chang, X. Zhang *et al.*, "Weight-sharing neural architecture search: A battle to shrink the optimization gap," *ACM Computing Surveys (CSUR)*, vol. 54, no. 9, pp. 1–37, 2021.

[39] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," 2019.

[40] S. You, C. Xu, C. Xu, and D. Tao, "Learning from multiple teacher networks," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 1285–1294.

[41] Y. Zhang, T. Xiang, T. M. Hospedales, and H. Lu, "Deep mutual learning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4320–4328.

[42] J. H. Cho and B. Hariharan, "On the efficacy of knowledge distillation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 4794–4802.

[43] C. Yang, L. Xie, S. Qiao, and A. L. Yuille, "Training deep neural networks in generations: A more tolerant teacher educates better students," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 5628–5635.

[44] Z. Chen, J. Niu, L. Xie, X. Liu, L. Wei, and Q. Tian, "Network adjustment: Channel search guided by flops utilization ratio," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 10 658–10 667.

[45] S. Guo, Y. Wang, Q. Li, and J. Yan, "Dmcp: Differentiable markov channel pruning for neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1539–1547.

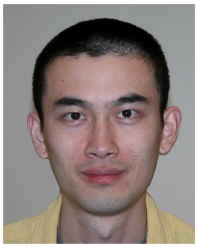
[46] X. Lu, H. Huang, W. Dong, X. Li, and G. Shi, "Beyond network pruning: a joint search-and-training approach," in *Proceedings of the Twenty-Ninth*

International Conference on International Joint Conferences on Artificial Intelligence, 2021, pp. 2583–2590.
[47] Y. Liu, W. Zhang, and J. Wang, "Adaptive multi-teacher multi-level knowledge distillation," *Neurocomputing*, vol. 415, pp. 106–113, 2020.
[48] Z. Yang, L. Shou, M. Gong, W. Lin, and D. Jiang, "Model compression with two-stage multi-teacher knowledge distillation for web question answering system," in *Proceedings of the 13th International Conference on Web Search and Data Mining*, 2020, pp. 690–698.
[49] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "Amc: Automl for model compression and acceleration on mobile devices," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 784–800.
[50] T. Wang, K. Wang, H. Cai, J. Lin, Z. Liu, H. Wang, Y. Lin, and S. Han, "Apq: Joint search for network architecture, pruning and quantization policy," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2078–2087.
[51] J. Ye, X. Lu, Z. Lin, and J. Z. Wang, "Rethinking the smaller-norm-less-informative assumption in channel pruning of convolution layers," *arXiv preprint arXiv:1802.00124*, 2018.
[52] K. Azarian, Y. S. Bhalgat, J. Lee, and T. Blankevoort, "Learned threshold pruning," 2021. [Online]. Available: https://openreview.net/forum?id=j0uePNuoBho
[53] T. Zhang, S. Ye, K. Zhang, J. Tang, W. Wen, M. Fardad, and Y. Wang, "A systematic dnn weight pruning framework using alternating direction method of multipliers," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 184–199.
[54] A. Kusupati, V. Ramanujan, R. Somani, M. Wortsman, P. Jain, S. Kakade, and A. Farhadi, "Soft threshold weight reparameterization for learnable sparsity," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5544–5555.
[55] K. Yu, C. Sciuto, M. Jaggi, C. Musat, and M. Salzmann, "Evaluating the search phase of neural architecture search," in *International Conference on Learning Representations*, 2019.
[56] X. Dai, P. Zhang, B. Wu, H. Yin, F. Sun, Y. Wang, M. Dukhan, Y. Hu, Y. Wu, Y. Jia *et al.*, "Chamnet: Towards efficient network design through platform-aware model adaptation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 11 398–11 407.
[57] S. Anwar and W. Sung, "Coarse pruning of convolutional neural networks with random masks," 2016.
[58] D. Mittal, S. Bhardwaj, M. M. Khapra, and B. Ravindran, "Studying the plasticity in deep convolutional neural networks using random pruning," *Machine Vision and Applications*, vol. 30, no. 2, pp. 203–216, 2019.
[59] L. Li and A. Talwalkar, "Random search and reproducibility for neural architecture search," in *Uncertainty in artificial intelligence*. PMLR, 2020, pp. 367–377.
[60] G. Adam and J. Lorraine, "Understanding neural architecture search techniques," *arXiv preprint arXiv:1904.00438*, 2019.
[61] P. Liashchynskyi and P. Liashchynskyi, "Grid search, random search, genetic algorithm: a big comparison for nas," *arXiv preprint arXiv:1912.06059*, 2019.
[62] X. Zhang, P. Hou, X. Zhang, and J. Sun, "Neural architecture search with random labels," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 10 907–10 916.
[63] A. Ashok, N. Rhinehart, F. Beainy, and K. M. Kitani, "N2n learning: Network to network compression via policy gradient reinforcement learning," *arXiv preprint arXiv:1709.06030*, 2017.
[64] M. Lin, R. Ji, Y. Wang, Y. Zhang, B. Zhang, Y. Tian, and L. Shao, "Hrank: Filter pruning using high-rank feature map," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1529–1538.
[65] Y. Liu, W. Zhang, and J. Wang, "Adaptive multi-teacher multi-level knowledge distillation," *Neurocomputing*, vol. 415, pp. 106–113, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231220311565
[66] Z. Yang, L. Shou, M. Gong, W. Lin, and D. Jiang, "Model compression with two-stage multi-teacher knowledge distillation for web question answering system." New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3336191.3371792
[67] M.-C. Wu, C.-T. Chiu, and K.-H. Wu, "Multi-teacher knowledge distillation for compressed video action recognition on deep neural networks," in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 2202–2206.
[68] Y. Hu, S. Sun, J. Li, X. Wang, and Q. Gu, "A novel channel pruning method for deep neural network compression," *arXiv preprint arXiv:1805.11394*, 2018.
[69] Z. Huang and N. Wang, "Data-driven sparse structure selection for deep neural networks," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 304–320.
[70] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient convnets," *arXiv preprint arXiv:1608.08710*, 2016.
[71] T.-W. Chin, R. Ding, C. Zhang, and D. Marculescu, "Towards efficient model compression via learned global ranking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1518–1528.

Xiaotong Lu received the B.S. degree in electronic engineering and the Ph. D. degree in circuits and System from the Xidian University, Xi'an, China, in 2016 and 2022, respectively. He is currently with the Sensetime Technology Co. His research interests include deep learning, compressive sensing, and image restoration.

Weisheng Dong (Member, IEEE) received the B.S. degree in electronic engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2004, and the Ph.D. degree in circuits and system from Xidian University, Xi'an, China, in 2010. He was a Visiting Student with Microsoft Research Asia, Beijing, China, in 2006. From 2009 to 2010, he was a Research Assistant with the Department of Computing, Hong Kong Polytechnic University, Hong Kong. In 2010, he joined the School of Electronic Engineering, Xidian University, as a Lecturer, where he has been a Professor since 2016. His research interests include deep learning, inverse problems in image processing, and sparse representation.

He was a recipient of the Best Paper Award at the SPIE Visual Communication and Image Processing (VCIP) in 2010. He has served as an associate editor of IEEE Transactions on Image Processing and is currently serving as an associate editor of SIAM Journal of Imaging Sciences.



Xin Li (Fellow, IEEE) received the B.S. (Hons.) degree in electronic engineering and information science from the University of Science and Technology of China, Hefei, China, in 1996, and the Ph.D. degree in electrical engineering from Princeton University, Princeton, NJ, USA, in 2000.

He was a Technical Staff Member with the Sharp Laboratories of America, Camas, WA, USA, from 2000 to 2002. Since 2003, he has been a Faculty Member with the Lane Department of Computer Science and Electrical Engineering. His research interests include image/video coding and processing. He was a recipient of the Best Student Paper Award at the Conference of Visual Communications and Image Processing in 2001, the Best Student Paper Award at the IEEE Asilomar Conference on Signals, Systems and Computers in 2006, and the Best Paper Award at the Conference of Visual Communications and Image Processing in 2010. He is currently serving as a member of the Image, Video and Multidimensional Signal Processing Technical Committee and an Associate Editor of the IEEE Transactions on Circuits and Systems for Video Technology.

Jinjian Wu (Member, IEEE) received the B.S. and Ph.D. degrees from Xidian University, Xi'an, China, in 2008 and 2013, respectively. From 2011 to 2013, he was a Research Assistant with Nanyang Technological University, Singapore, where he was a Postdoctoral Research Fellow from 2013 to 2014. From 2015 to 2019, he was an Associate Professor with Xidian University, where he has been a Professor since 2019. His research interests include visual perceptual modeling, biomimetic imaging, quality evaluation, and object detection. He received the Best Student Paper Award at ISCAS in 2013. He has served as an Associate Editor for the journal of Circuits, Systems, and Signal Processing, the Special Section Chair for IEEE Visual Communications and Image Processing in 2017, and the Section Chair/an Organizer/a TPC Member for ICME from 2014 to 2015, PCM from 2015 to 2016, ICIP in 2015, VCIP in 2018, and AAAI in 2019.

Leida Li (Member, IEEE) received the B.S. and Ph.D. degrees from Xidian University, Xi'an, China, in 2004 and 2009, respectively. In 2008, he was a Research Assistant with the Department of Electronic Engineering, Kaohsiung University of Science and Technology, Kaohsiung, Taiwan. From 2014 to 2015, he was a Visiting Research Fellow with the Rapid-Rich Object Search (ROSE) Lab, School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, where he was a Senior Research Fellow from 2016 to 2017. He is currently a Professor with the School of Artificial Intelligence, Xidian University. His research interests include multimedia quality assessment, affective computing, information hiding, and image forensics. He has served as an SPC for IJCAI from 2019 to 2021, the Session Chair for ICMR in 2019 and PCM in 2015, and a TPC Member for CVPR in 2021, ICCV in 2021, AAAI from 2019 to 2021, ACM MM from 2019 to 2020, ACM MM-Asia in 2019, ACII in 2019, and PCM in 2016. He is also an Associate Editor of the Journal of Visual Communication and Image Representation and the EURASIP Journal on Image and Video Processing.

Guangming Shi (Fellow, IEEE) received the B.S. degree in Automatic Control in 1985, the M.S. degree in Computer Control and Ph.D. degree in Electronic Information Technology, all from Xidian University in 1988 and 2002, respectively.

He joined the School of Electronic Engineering, Xidian University, in 1988. From 1994 to 1996, as a Research Assistant, he cooperated with the Department of Electronic Engineering at the University of Hong Kong. Since 2003, he has been a Professor in the School of Electronic Engineering at Xidian University, and in 2004 the head of National Instruction Base of Electrician and Electronic (NIBEE). From June to December in 2004, he had studied in the Department of Electronic Engineering at University of Illinois at Urbana-Champaign (UIUC). He is currently the vice president of Xidian University, and the academic leader in the subject of Circuits and Systems. His research interests include compressed sensing, theory and design of multirate filter banks, image denoising, low-bit-rate image/video coding and implementation of algorithms for intelligent signal processing (using DSP and FPGA). He has authored or co-authored over 60 research papers.